

Embedded Systems mit RISC-V

5. Digitale Ein-/Ausgabe

V1.0, Patrick Ritschel



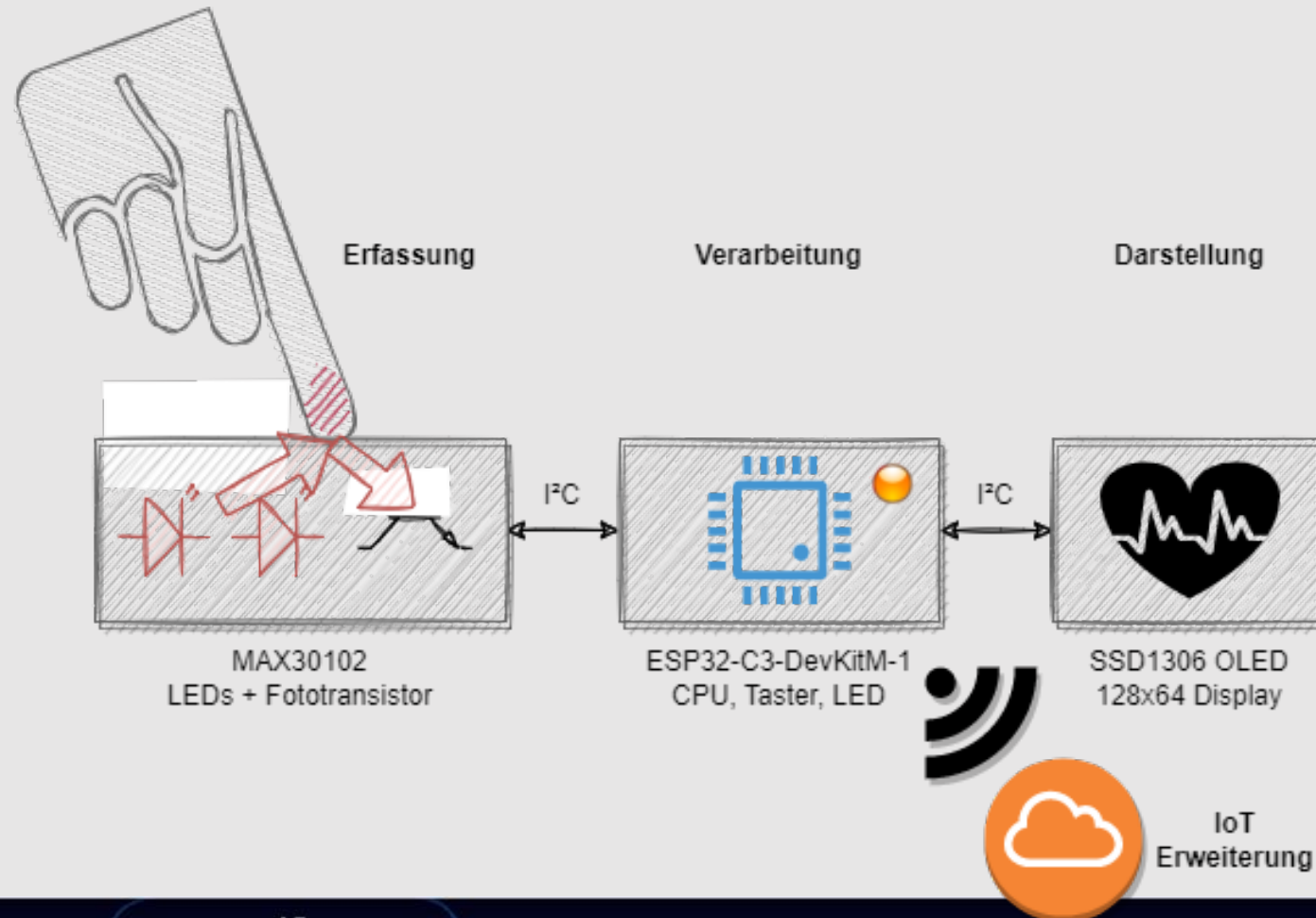
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    float y = 0.0f;  
    for (int i = 0; i < pFilter->order; i++)  
        y += pFilter->coefficients[i] * value;  
    return y;  
}
```

Buchprojekt

Pulsoximeter



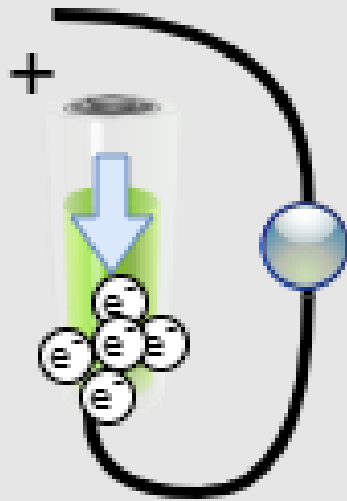
IIRFilter

Filter

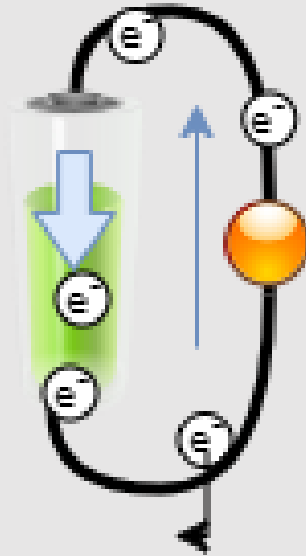
```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Elektrotechnische Grundlagen

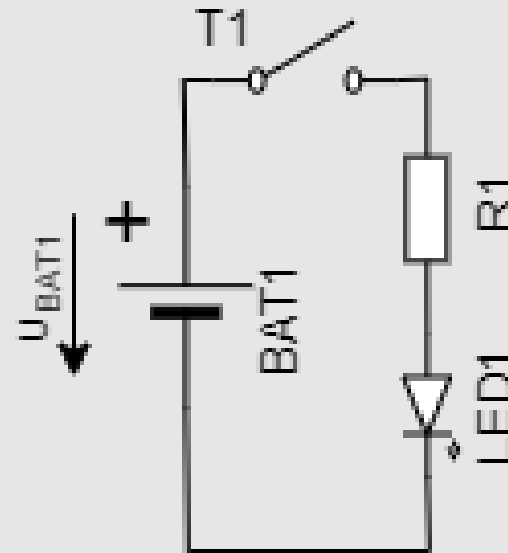
Einfacher Schaltkreis mit Batterie und LED



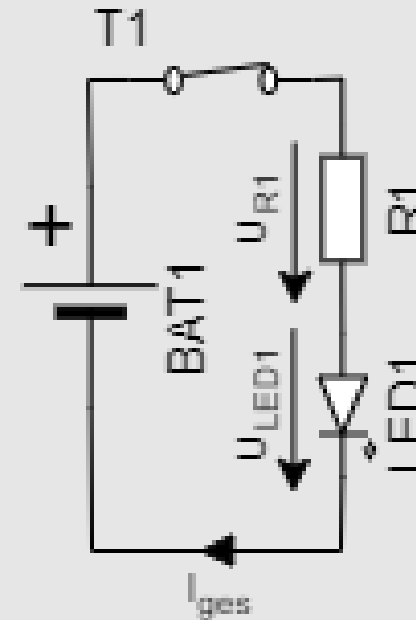
a)



b)



c)



d)

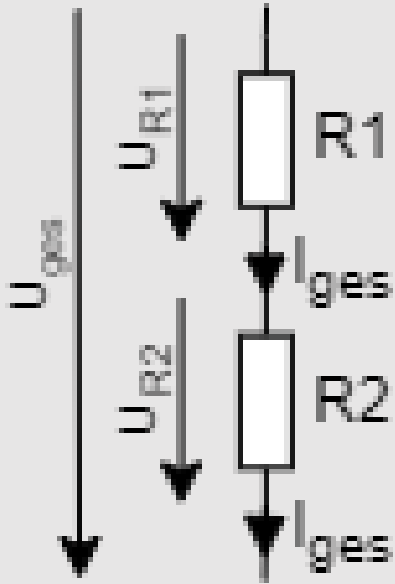
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

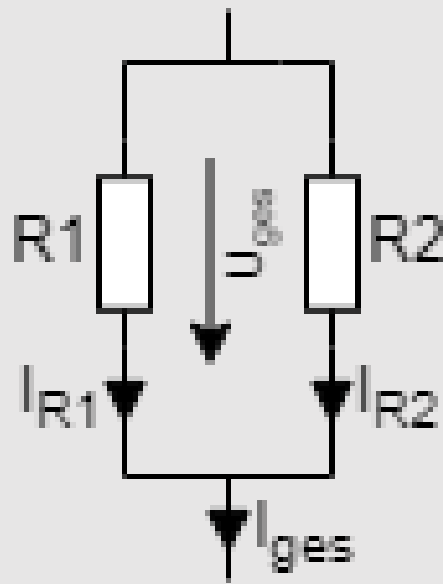
Elektrotechnische Grundlagen

Kirchhoff und Ohm



Serienschaltung

$$U_{\text{ges}} = U_{R1} + U_{R2}$$



Parallelschaltung

$$I_{\text{ges}} = I_{R1} + I_{R2}$$

$$I[A] = \frac{U[V]}{R[\Omega]} \Leftrightarrow U = R \cdot I \Leftrightarrow R = \frac{U}{I}$$

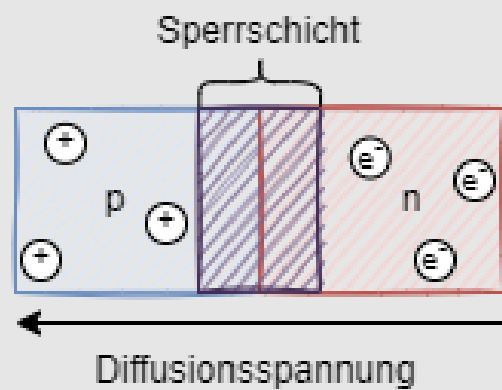
IIRFilter

Filter

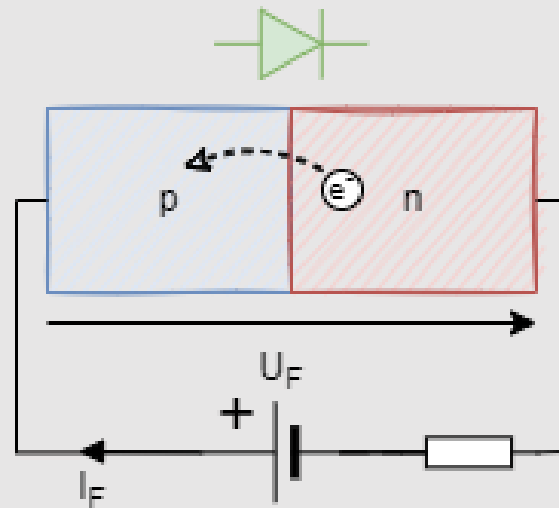
```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Elektrotechnische Grundlagen

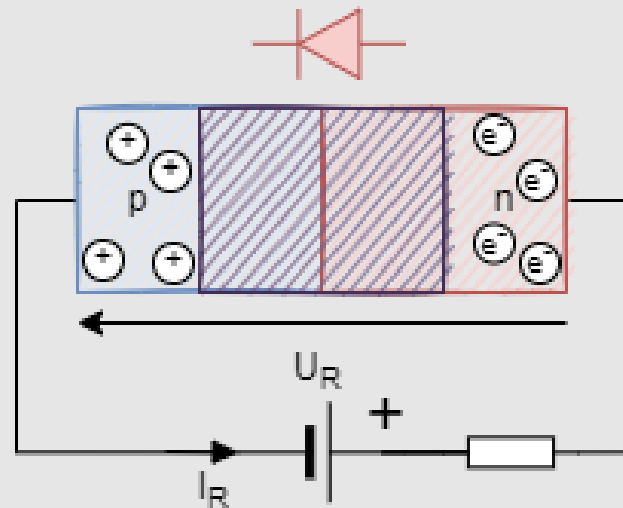
Diode



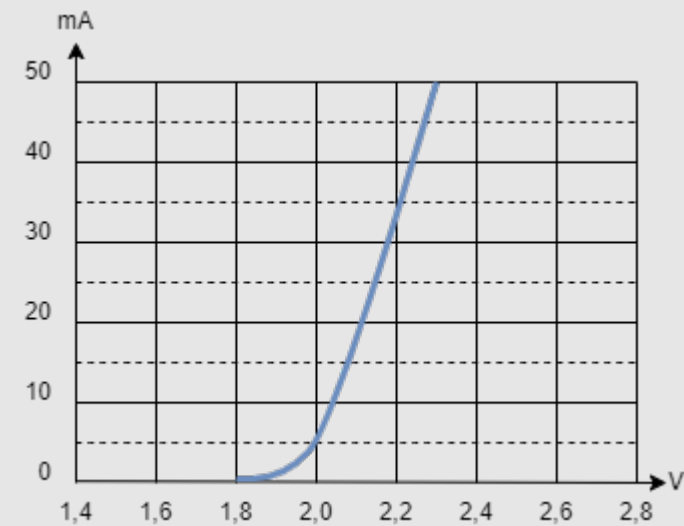
a) p-n-Übergang unbeschaltet



b) Diode in Durchlassrichtung



c) Diode in Sperrrichtung



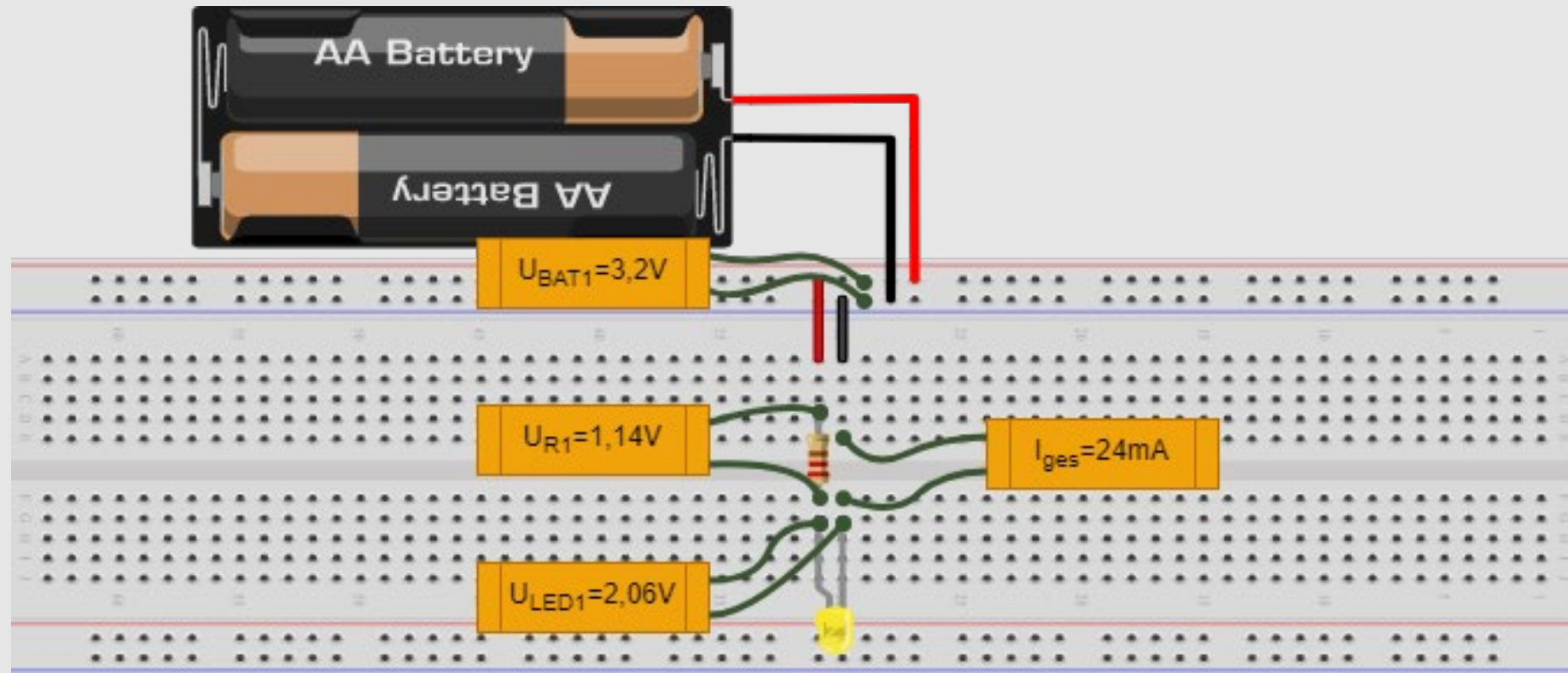
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Elektrotechnische Grundlagen

Aufbau mit Steckplatine („Breadboard“)



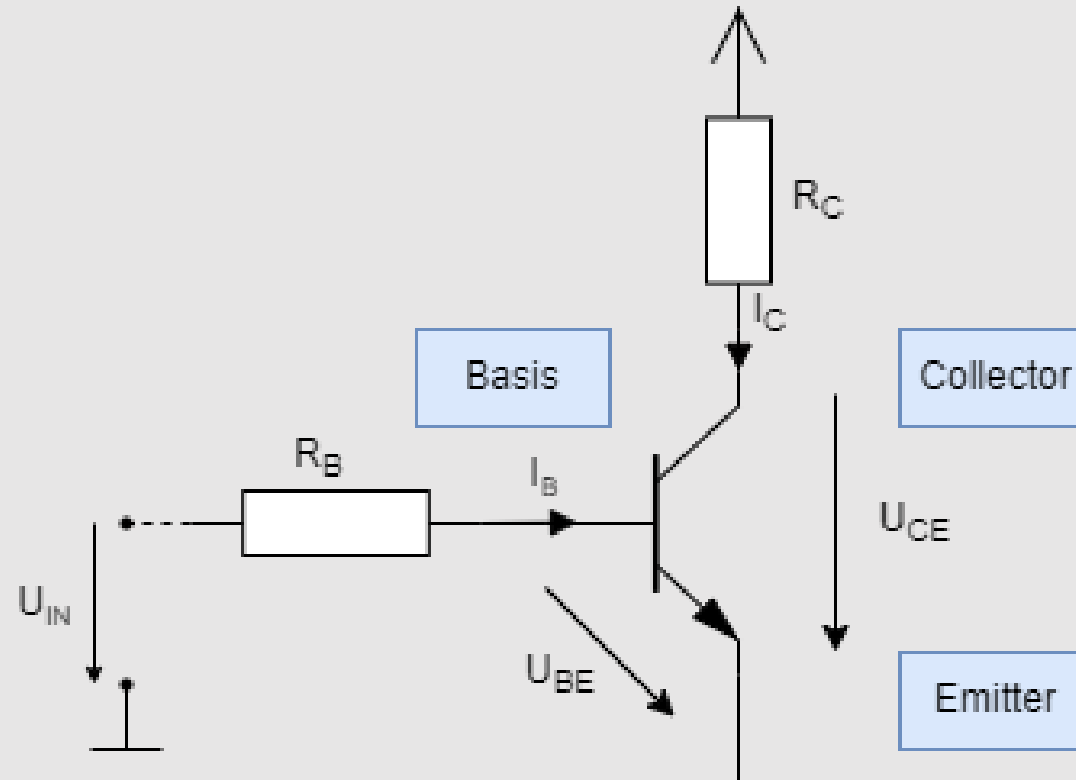
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Elektrotechnische Grundlagen

NPN-Transistor in Emitterschaltung



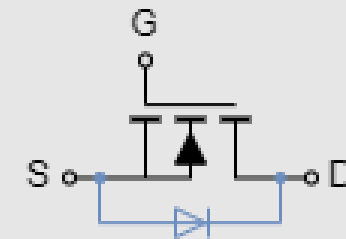
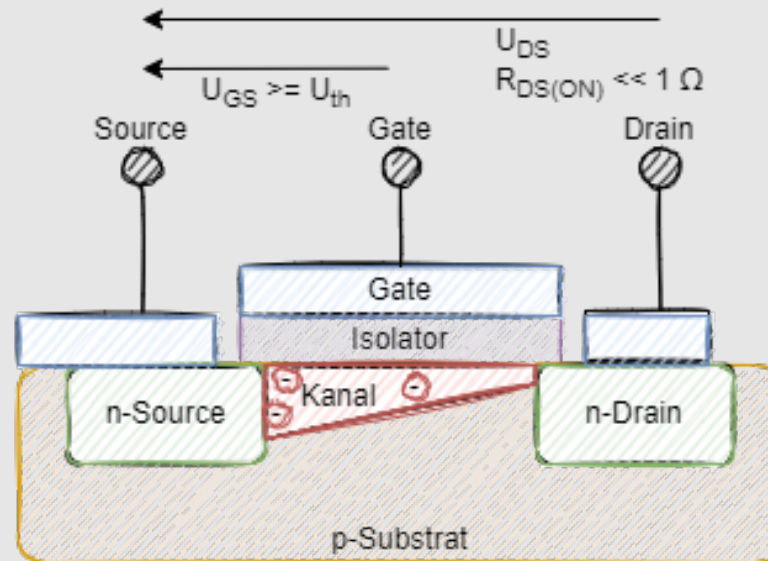
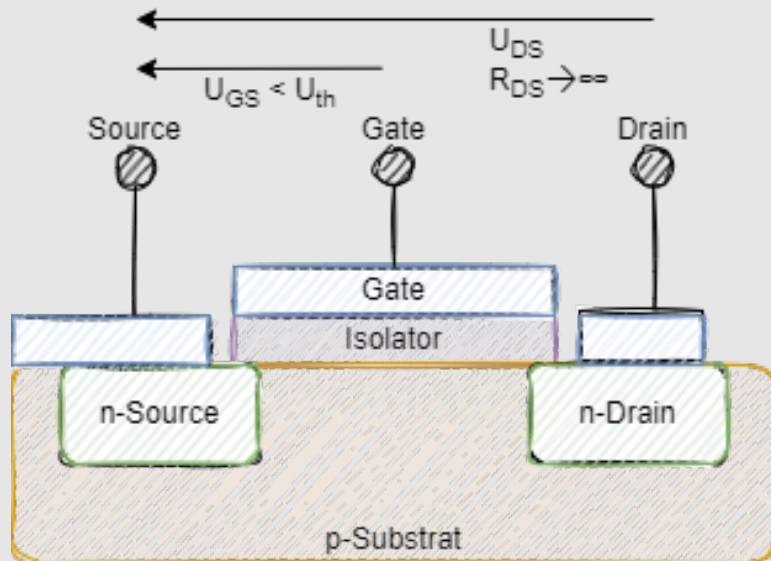
IIRFilter

Filter

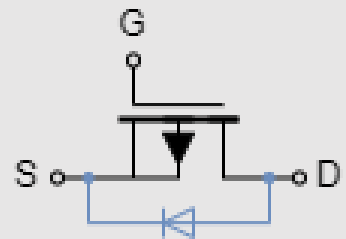
```
float iirfilter_filterValue(Filter* pFilter, float value) {
```


Elektrotechnische Grundlagen

MOSFET Aufbau und Typen



n-Kanal-MOSFET
Anreicherungstyp,
selbstsperrend



p-Kanal-MOSFET
Verarmungstyp,
selbstleitend

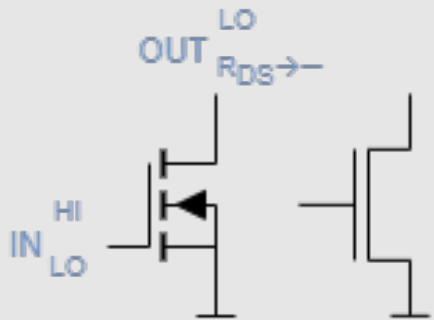
IIRFilter

Filter

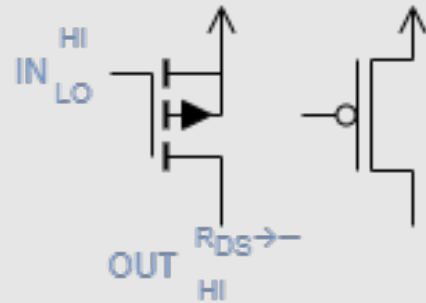
```
float iirfilter_filterValue(Filter* pFilter, float value) {
```


Elektrotechnische Grundlagen

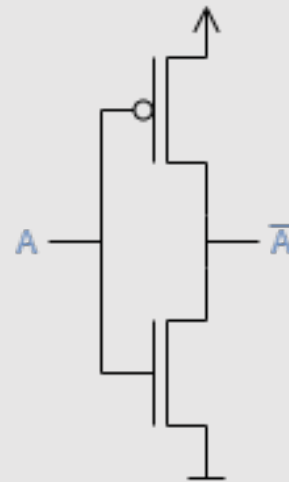
Switches und Gatter in CMOS



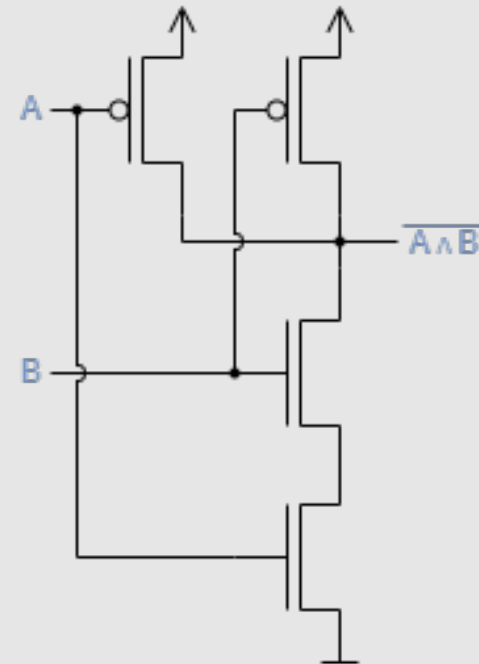
NMOS Low Side Switch



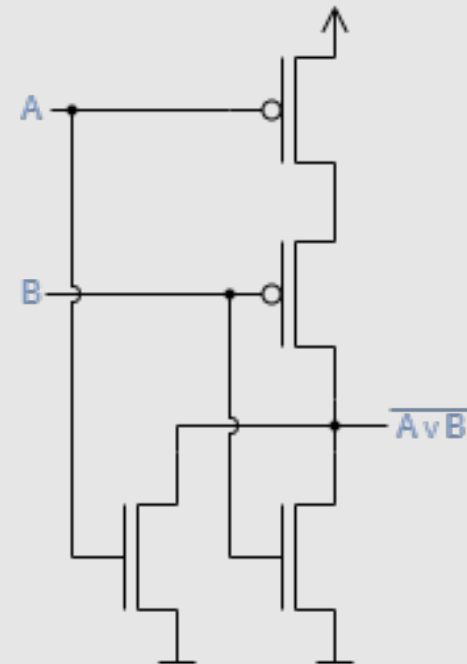
NMOS High Side Switch



CMOS-Inverter



NAND-Gatter



NOR-Gatter

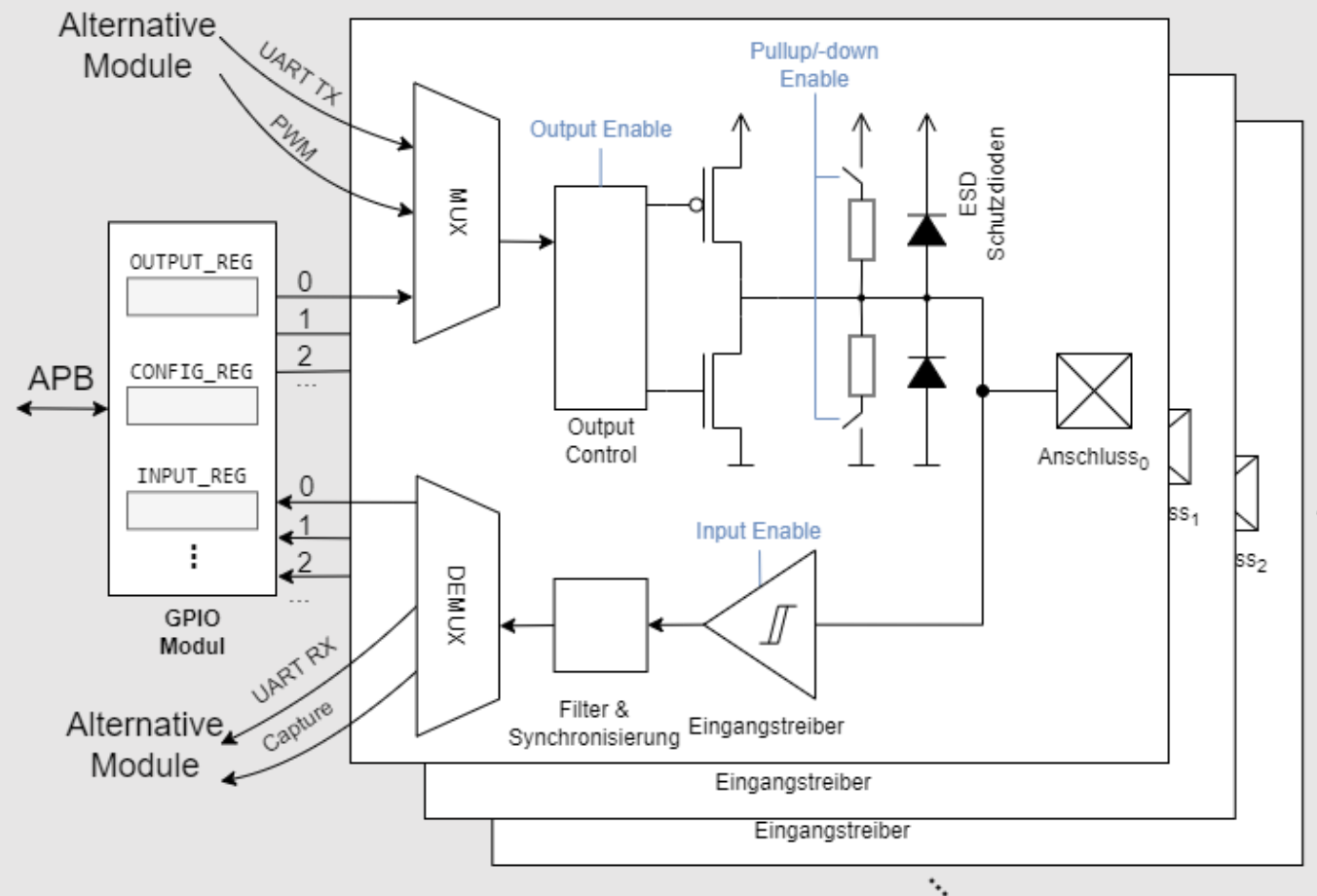
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

GPIO-Modul



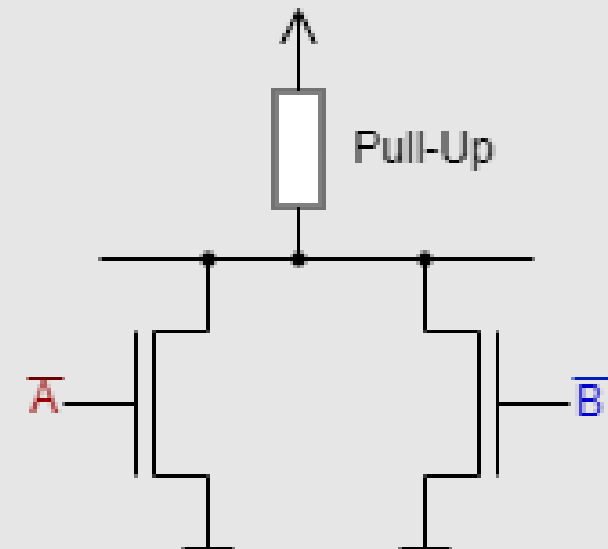
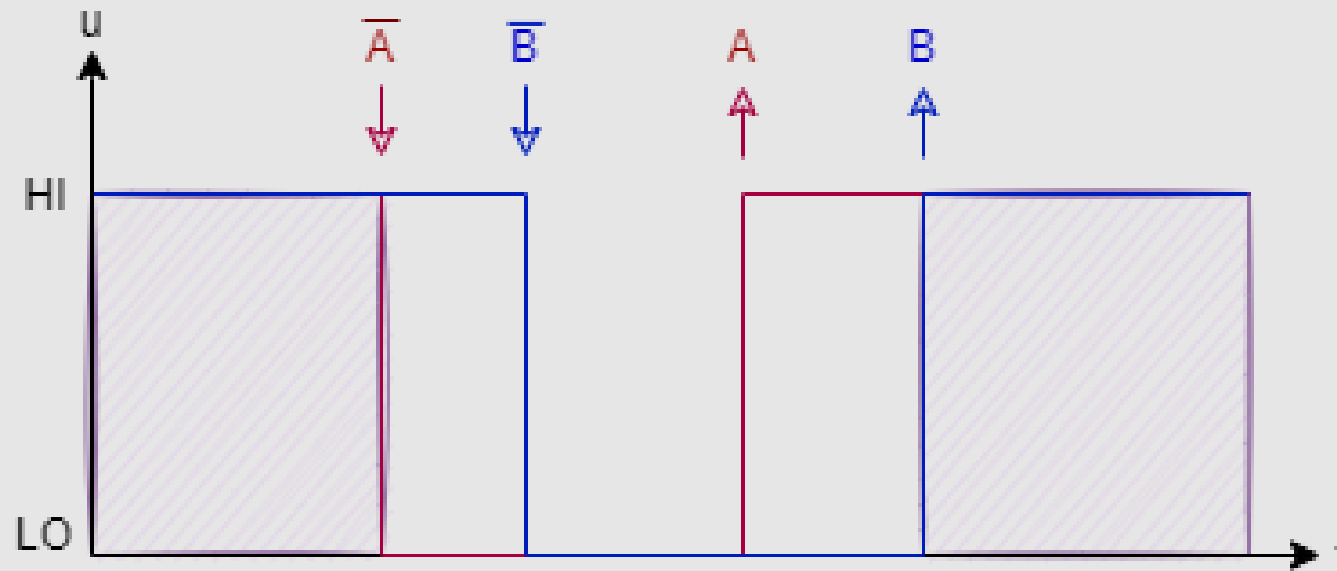
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

Wired AND



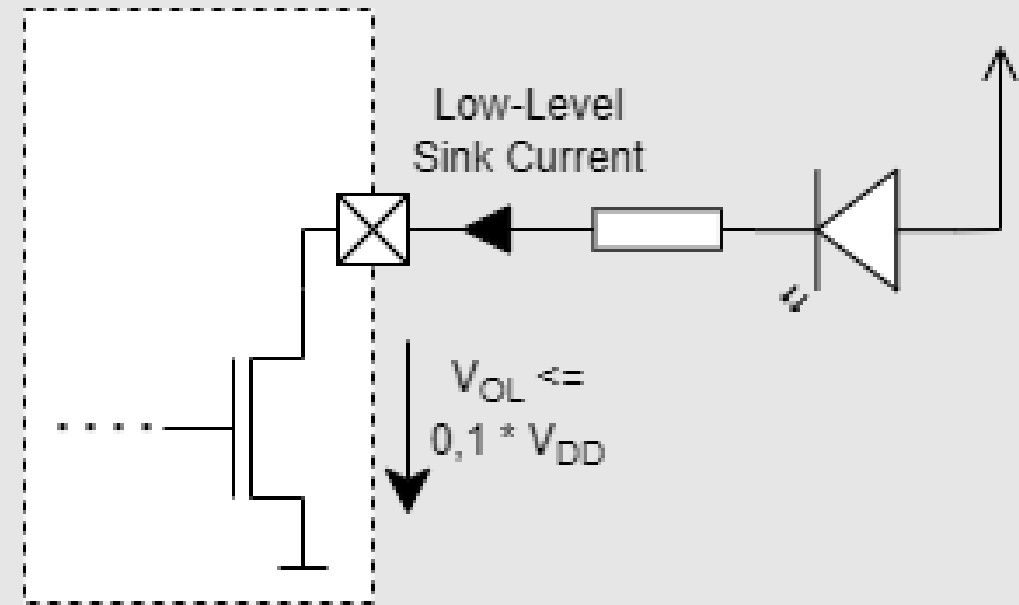
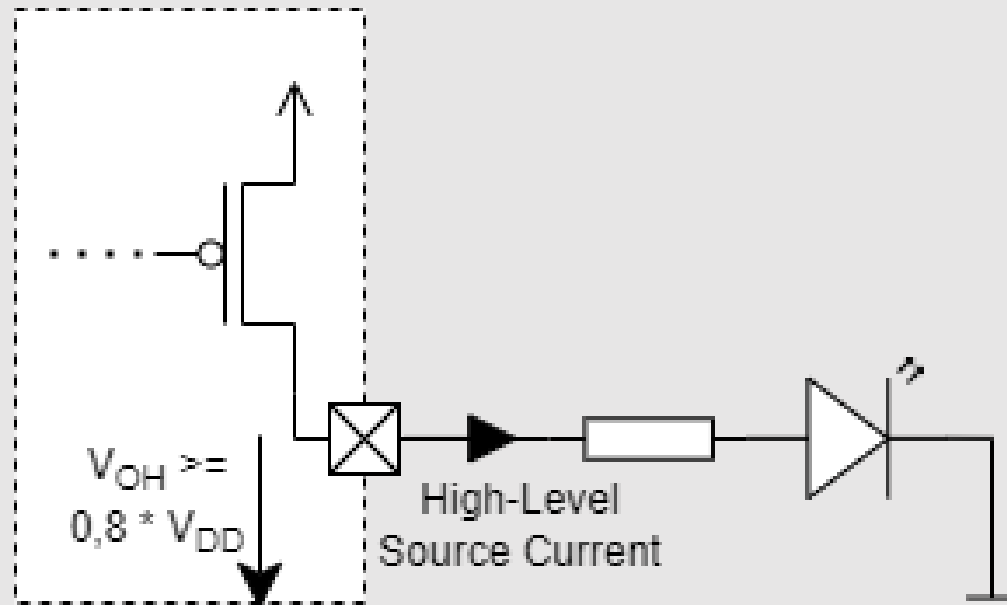
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

GPIO als Quelle und Senke



IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

GPIO per direktem Registerzugriff



```
#define GPIO_BASE          0x60004000
#define GPIO_ENABLE_REG_OFFS 0x0020
#define gpioEnableReg *((volatile uint32_t*)
    ↪ (GPIO_BASE | GPIO_ENABLE_REG_OFFS))
```

```
gpioEnableReg |= (1 << GPIO_NUM_4) | (1 << GPIO_NUM_5);
```

```
#define LED1_GPIO    GPIO_NUM_4
#define LED2_GPIO    GPIO_NUM_5
```

```
#define GPIO_OUT_REG_OFFS    0x0004
#define gpioOutReg *((volatile uint32_t*)
    ↪ (GPIO_BASE | GPIO_OUT_REG_OFFS))
```

```
while (true) {
    gpioOutReg ^= (1 << LED1_GPIO) | (1 << LED2_GPIO);
    vTaskDelay(1000 / portTICK_PERIOD_MS);
}
```

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

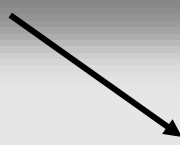
Einstellung des PIN Multiplexings



```
#define IO_MUX_BASE 0x60009000
#define IO_MUX_GPIOn_REG_OFFSETS(n) (0x0004 + n * 4)
#define iomuxGPIO4Reg *((volatile uint32_t*)
    ↪ (IO_MUX_BASE | IO_MUX_GPIOn_REG_OFFSETS(4)))
```

```
#define IO_MUX_GPIOn_FUN_DRV 10
#define IO_MUX_GPIOn_MCU_SEL 12
#define IO_MUX_PIN_FUNC_GPIO 1
#define IO_MUX_PIN_DRV_40mA 3
```

```
iomuxGPIO4Reg &= ~(0x7 << IO_MUX_GPIOn_MCU_SEL);
iomuxGPIO4Reg |= (IO_MUX_PIN_FUNC_GPIO << IO_MUX_GPIOn_MCU_SEL);
iomuxGPIO4Reg &= ~(0x3 << IO_MUX_GPIOn_FUN_DRV);
iomuxGPIO4Reg |= (IO_MUX_PIN_DRV_40mA << IO_MUX_GPIOn_FUN_DRV);
```



```
uint32_t reg = (iomuxGPIO4Reg &
    ↪ ~((0x7 << IO_MUX_GPIOn_MCU_SEL) |
    ↪ (0x3 << IO_MUX_GPIOn_FUN_DRV)));
iomuxGPIO4Reg = (reg |
    ↪ ((IO_MUX_PIN_FUNC_GPIO << IO_MUX_GPIOn_MCU_SEL) |
    ↪ (IO_MUX_PIN_DRV_40mA << IO_MUX_GPIOn_FUN_DRV)));
```

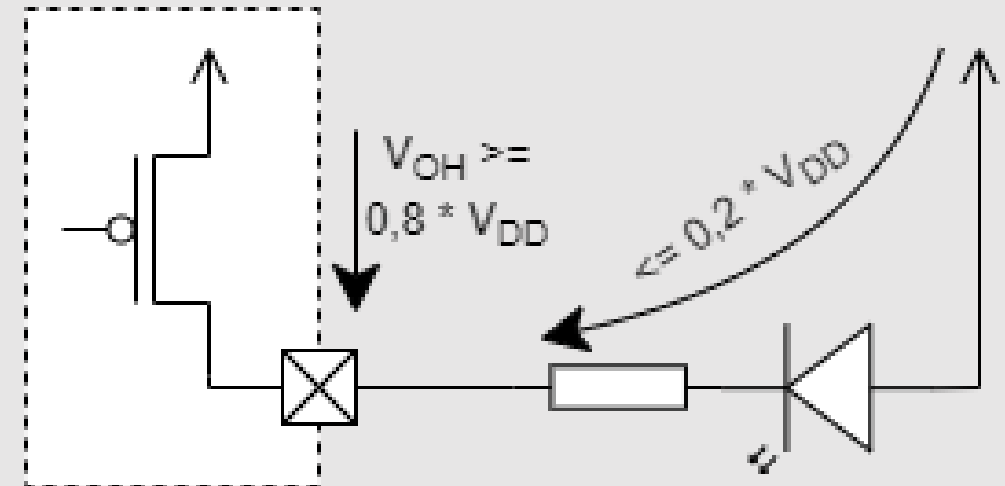
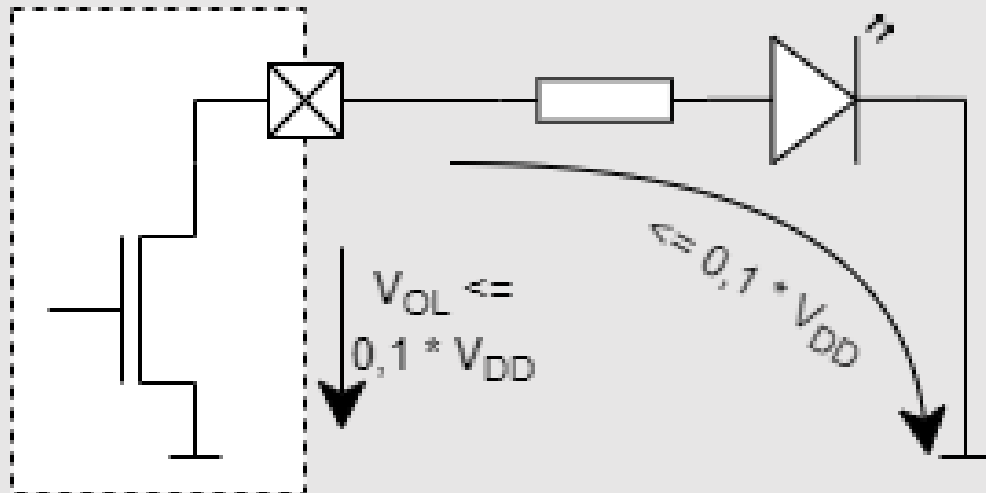
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```


Digital I/O

Push-/Pull Spannungen, abgeschaltet



IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

Set-/Reset-Register vs. Read/Modify/Write



Direkter Zugriff	Read/Modify/Write
<code>GPIO_OUT_W1TS_REG = 0x17</code>	<code>GPIO_OUT_REG = 0x17</code>
<code>GPIO_OUT_W1TC_REG = 0x17</code>	<code>GPIO_OUT_REG &= ~0x17</code>

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

Bitfielddefinition



```
struct IoMuxGpioReg {  
    uint32_t ioMuxGpioMcuOe : 1; // bit0  
    uint32_t ioMuxGpioSlpSel : 1; // bit1  
    uint32_t ioMuxGpioMcuWpd : 1; // bit2  
    uint32_t ioMuxGpioMcuWpu : 1; // bit3  
    uint32_t ioMuxGpioMcuIe : 1; // bit4  
    uint32_t reserved0 : 2; // bit5..6  
    uint32_t ioMuxGpioFunWpd : 1; // bit7  
    uint32_t ioMuxGpioFunWpu : 1; // bit8  
    uint32_t ioMuxGpioFunIe : 1; // bit9  
    uint32_t ioMuxGpioFunDrv : 2; // bit10..11  
    uint32_t ioMuxGpioMcuSel : 3; // bit12..14  
    uint32_t ioMuxGpioFilterEn : 1; // bit15  
    uint32_t : 16; // bit16..31  
};
```

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    [ ... ]  
}
```

Digital I/O

Zugriff per Bitfield



```
volatile struct IoMuxGpioReg* pIoMuxGpio4Reg =  
    ↪ (volatile struct IoMuxGpioReg*)  
    ↪ (IO_MUX_BASE | IO_MUX_GPIOn_REG_OFFSET(4));
```

```
pIoMuxGpio4Reg->ioMuxGpioMcuSel = IO_MUX_PIN_FUNC_GPIO;  
pIoMuxGpio4Reg->ioMuxGpioFunDrv = IO_MUX_PIN_DRV_40mA;
```

```
#define ioMuxGpio5Reg (*(volatile struct IoMuxGpioReg*)  
    ↪ (IO_MUX_BASE | IO_MUX_GPIOn_REG_OFFSET(5)))
```

```
ioMuxGpio5Reg.ioMuxGpioMcuSel = IO_MUX_PIN_FUNC_GPIO;  
ioMuxGpio5Reg.ioMuxGpioFunDrv = IO_MUX_PIN_DRV_40mA;
```

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

Union für Raw-Zugriff



```
union IoMuxGpioReg {  
    struct {  
        uint32_t ioMuxGpioMcu0e : 1;  
        // [...]  
    };  
    uint32_t rawValue;  
};
```

```
pIoMuxGpio4Reg->ioMuxGpioMcuSel = ...
```

```
pIoMuxGpio4Reg->rawValue = (... | ...)
```

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    [ ... ]  
}
```


Digital I/O

Zugriff per API des ESP-IDF



```
gpio_config_t gpioConfig = {  
    .pin_bit_mask = (1 << LED1_GPIO) | (1 << LED2_GPIO),  
    .mode = GPIO_MODE_OUTPUT,  
    .pull_up_en = false,  
    .pull_down_en = false,  
    .intr_type = GPIO_INTR_DISABLE  
};  
gpio_config(&gpioConfig);
```

```
uint32_t level = 0;  
while (true) {  
    level = !level;  
    gpio_set_level(LED1_GPIO, level);  
    gpio_set_level(LED2_GPIO, level);  
    vTaskDelay(1000 / portTICK_PERIOD_MS);  
}
```

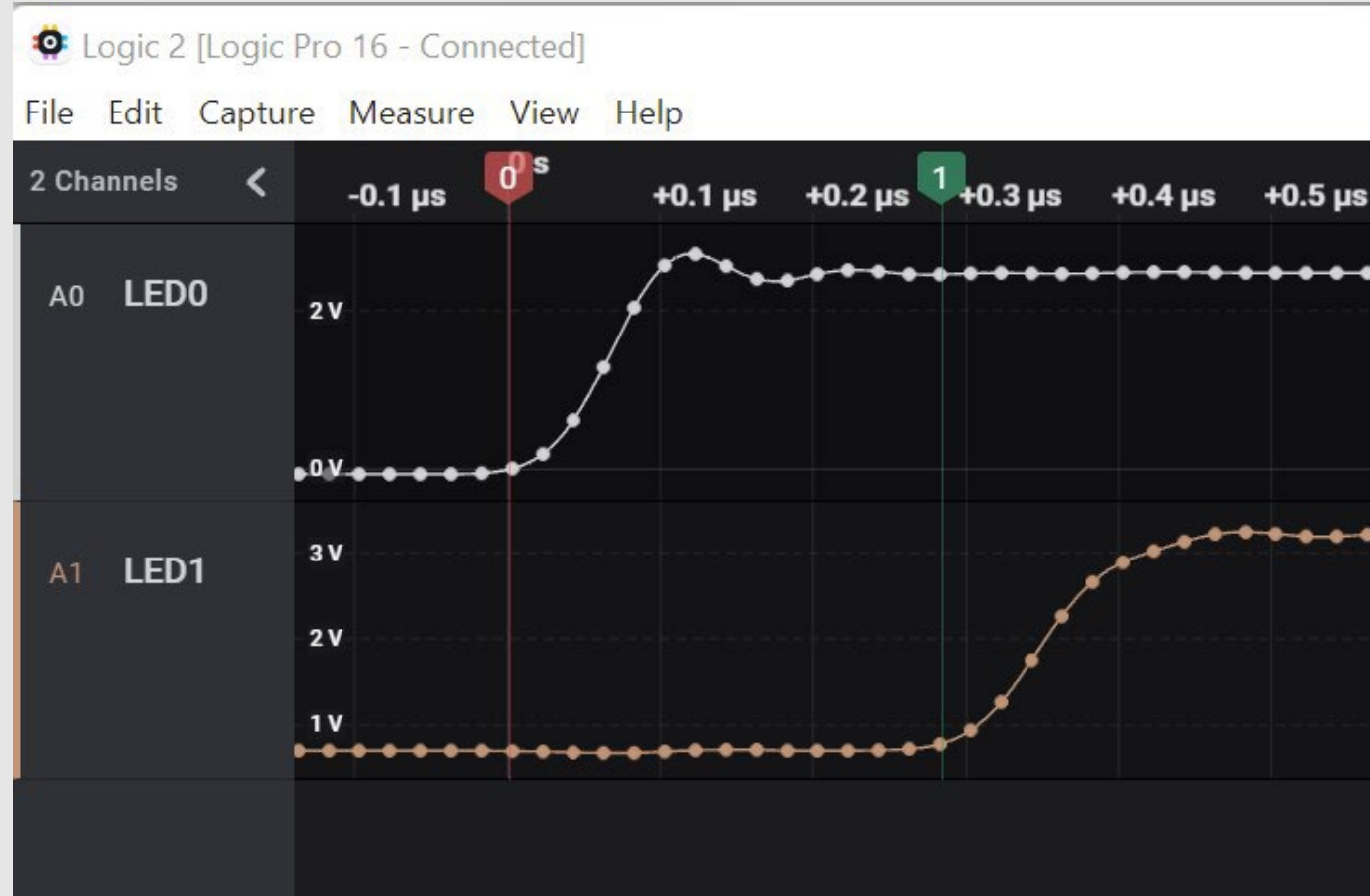
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {  
    [ ... ]  
}
```

Digital I/O

LEDs schalten hintereinander



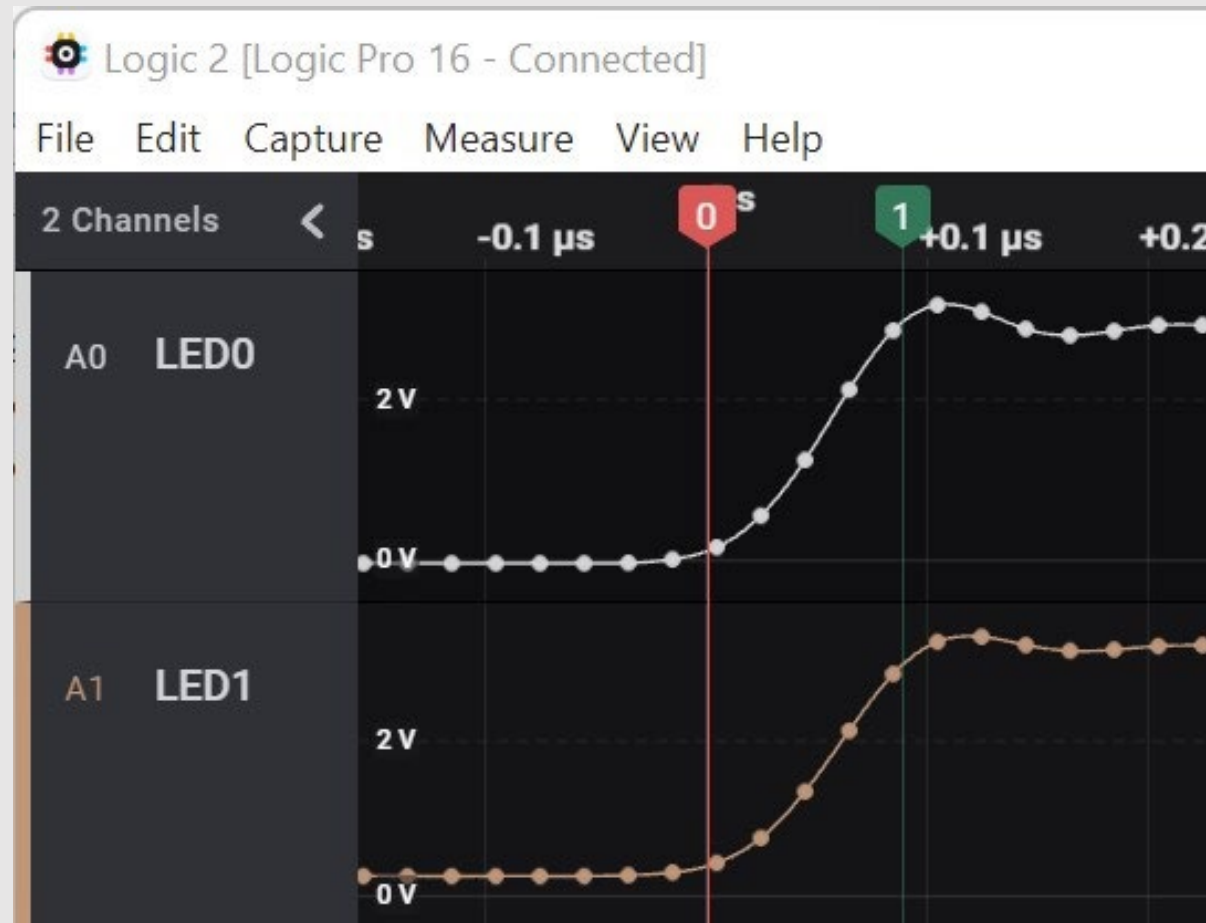
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```


Digital I/O

LEDs schalten gleichzeitig



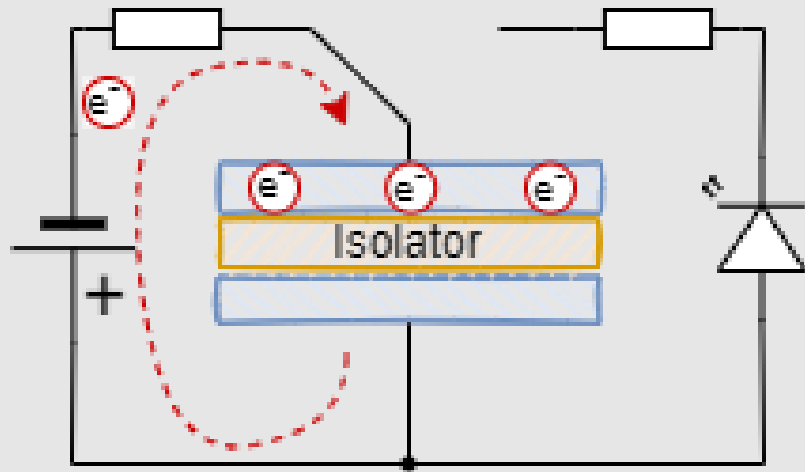
IIRFilter

Filter

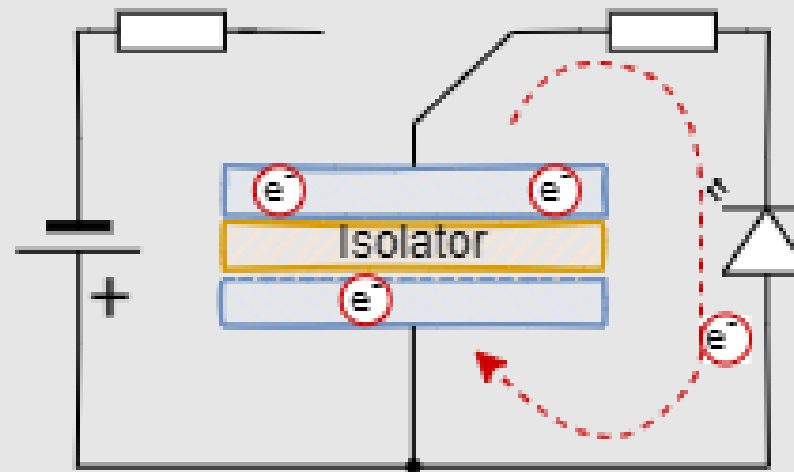
```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Elektrische Grundlagen

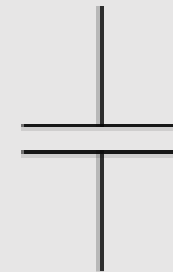
Kondensator



a) Laden des
Kondensators



b) Entladen des
Kondensators



c) Schaltplan-
symbol

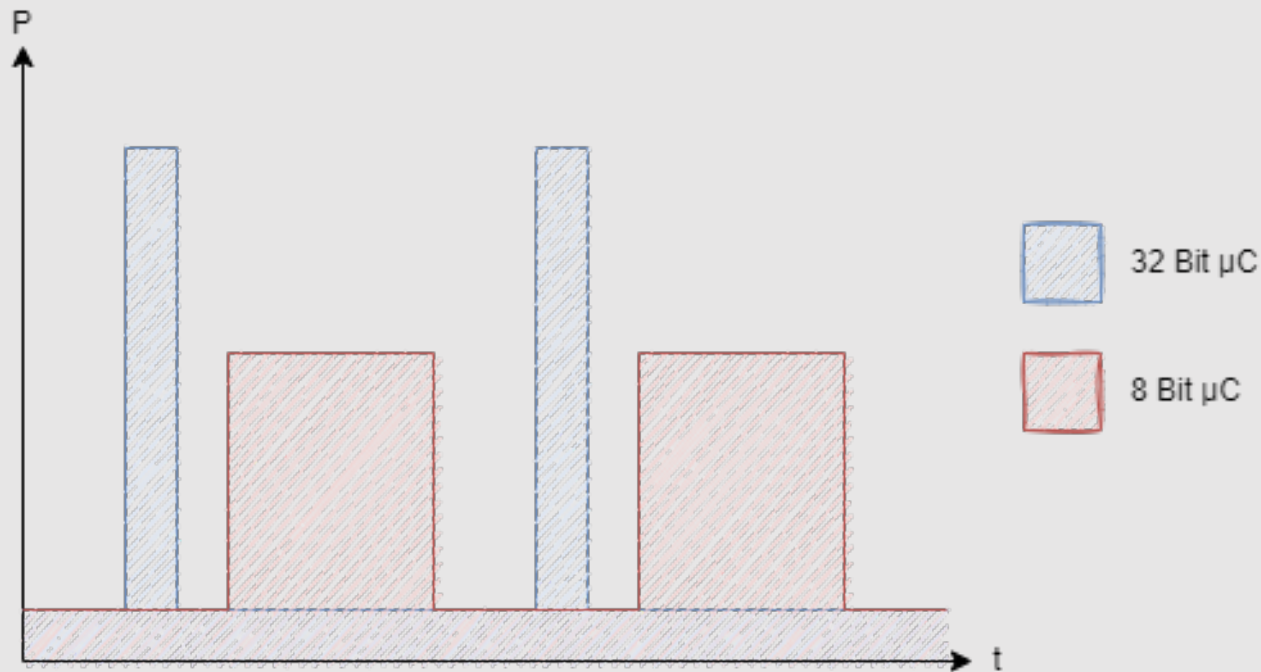
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Elektrische Grundlagen

Leistungsaufnahme



$$W = Q \cdot U$$

$$W = I \cdot U \cdot t$$

$$P = U \cdot I$$

$$W = \int_{t=t_0}^{t_1} u(t) \cdot i(t) dt$$

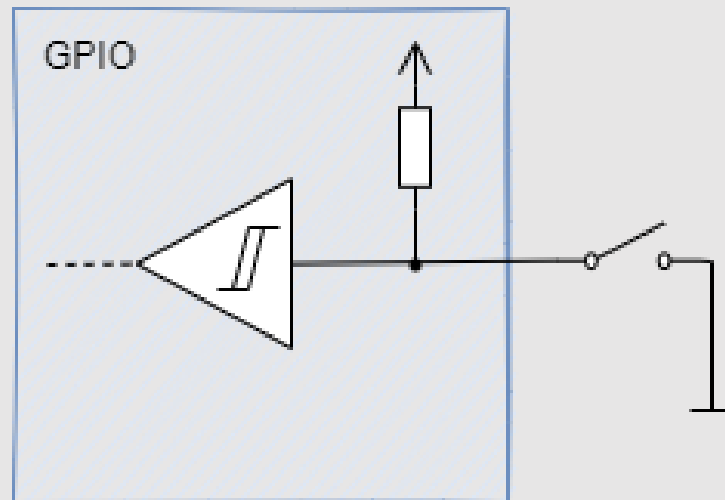
IIRFilter

Filter

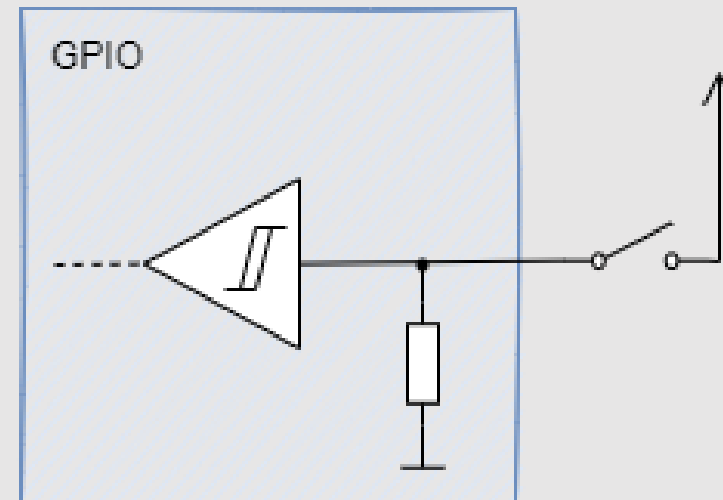
```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

Taster anschließen



a) Active-low-Konfiguration



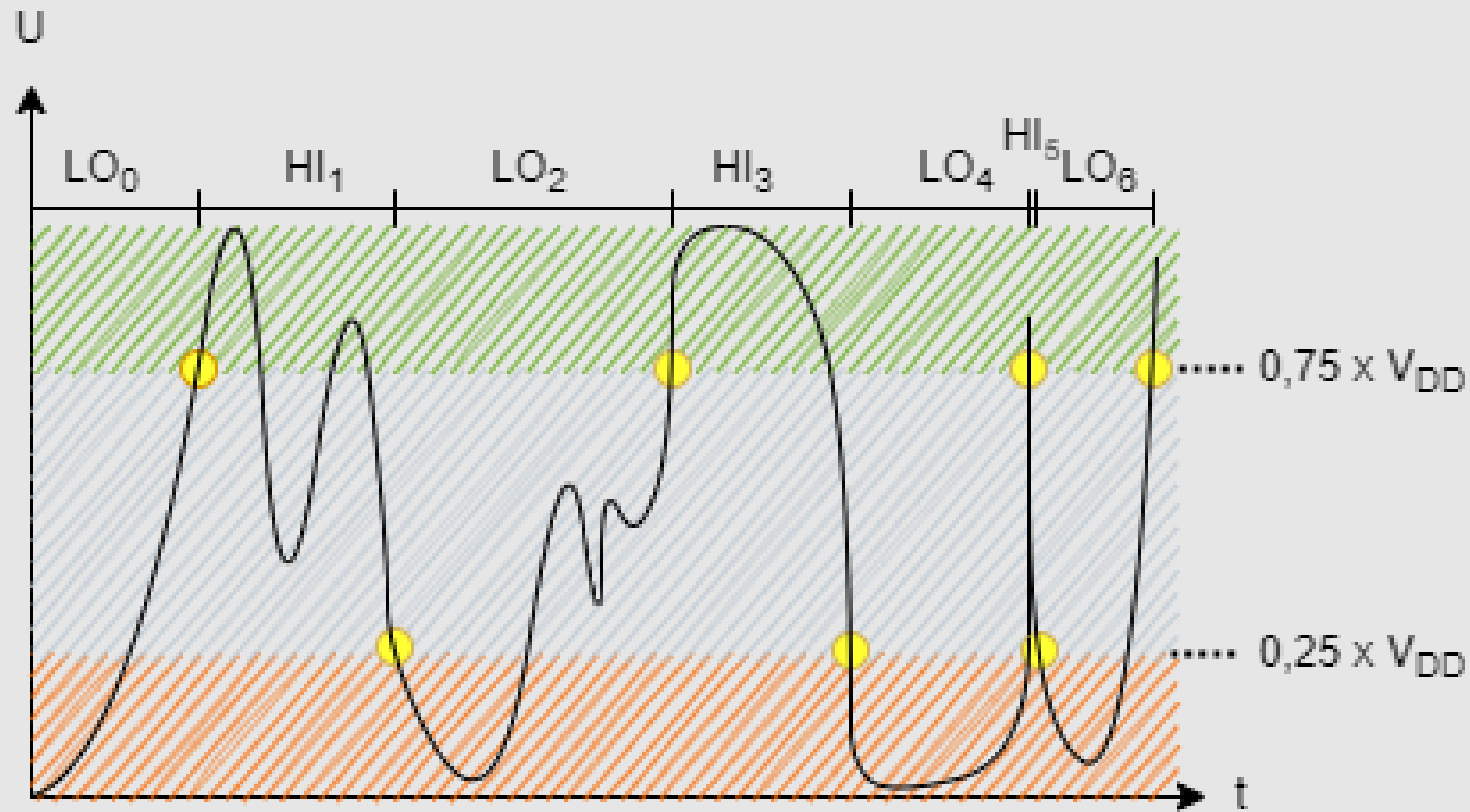
b) Active-high-Konfiguration

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O Schmitt-Trigger

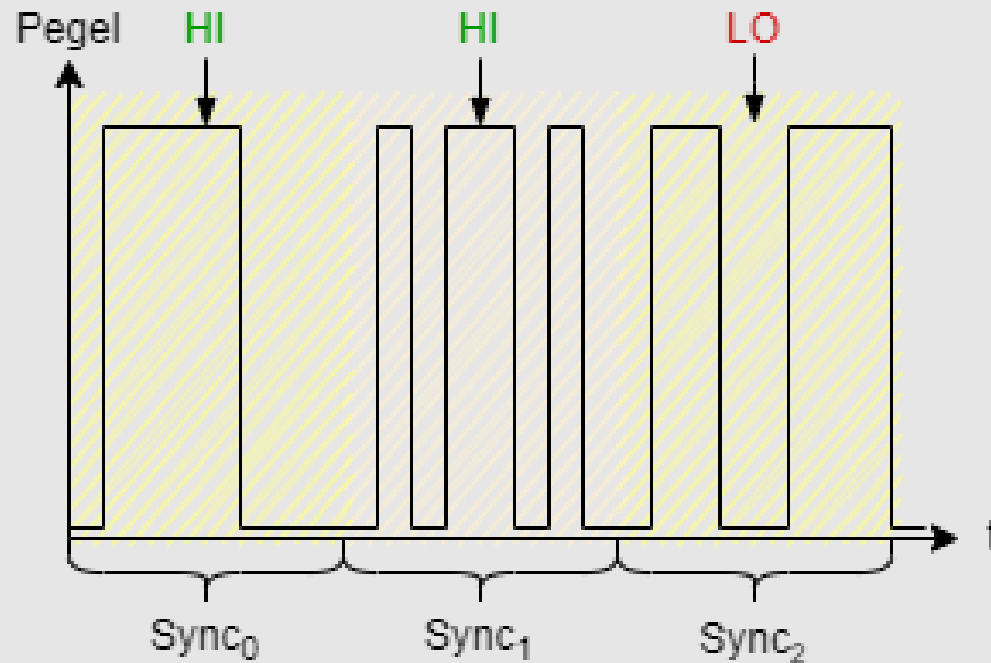


IIRFilter

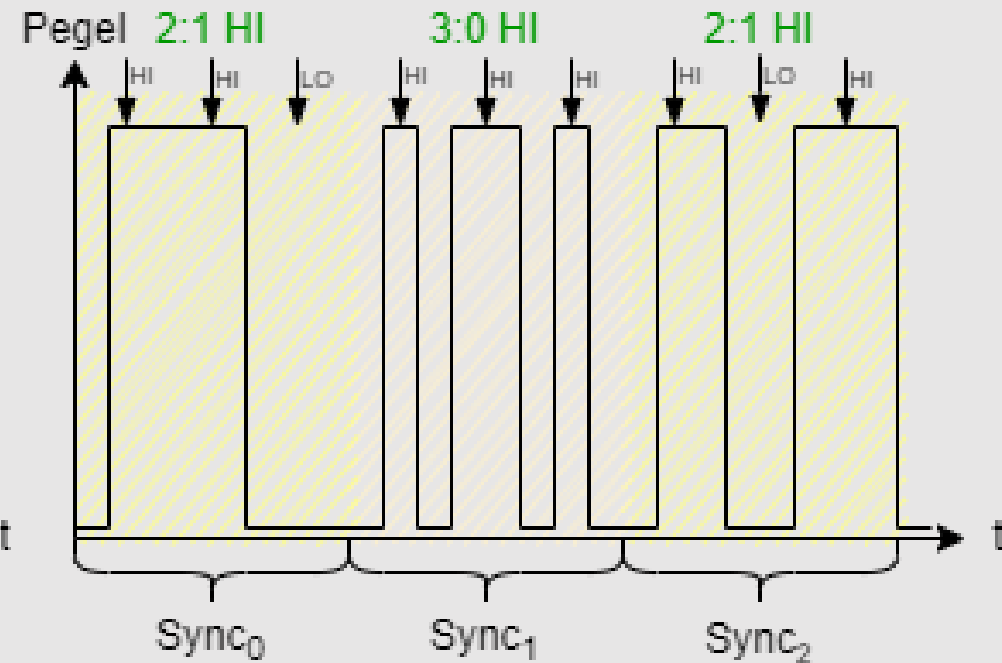
Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O Oversampling



a) Einfaches Oversampling



b) Dreifaches Oversampling



```
float iirfilter_filterValue(Filter* pFilter, float value) {
```


Digital I/O

Taster: Zustand einlesen



```
gpio_config_t gpioConfigIn = {
    .pin_bit_mask = (1 << BTN_GPIO),
    .mode = GPIO_MODE_INPUT,
    .pull_up_en = true,
    .pull_down_en = false,
    .intr_type = GPIO_INTR_DISABLE
};
gpio_config(&gpioConfigIn);
```

```
uint32_t ledLevel = 0;
uint32_t btnLevel = 1; // released@active-low
while (true) {
    if (gpio_get_level(BTN_GPIO) != btnLevel) {
        btnLevel = gpio_get_level(BTN_GPIO);
        if (btnLevel == 0) { // pressed
            ledLevel = !ledLevel;
            gpio_set_level(LED1_GPIO, ledLevel);
            gpio_set_level(LED2_GPIO, ledLevel);
        }
    }
}
```

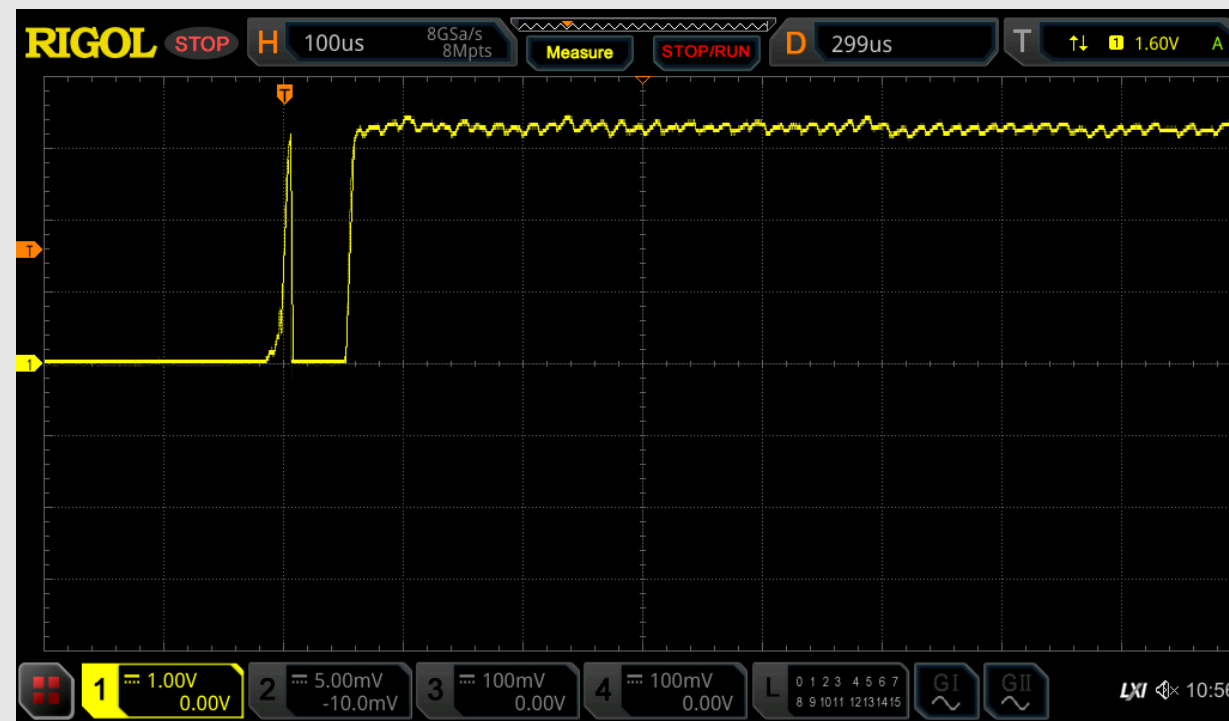
IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

Prellen des Tasters



IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```

Digital I/O

Entprellen des Tasters



```
uint32_t ledLevel = 0;
uint32_t btnActive = 0;
while (true) {
    if (gpio_get_level(BTN_GPIO) == 0) { // pressed
        if (++btnActive == 2) { // pressed long enough
            ledLevel = !ledLevel;
            gpio_set_level(LED1_GPIO, ledLevel);
            gpio_set_level(LED2_GPIO, ledLevel);
        }
    } else {
        btnActive = 0;
    }
    vTaskDelay(50 / portTICK_PERIOD_MS);
}
```

IIRFilter

Filter

```
float iirfilter_filterValue(Filter* pFilter, float value) {
```